

Leant2: Answers to the Expert Questions

From further-improvement proposals to a sound,
reproducible synthesis architecture

Prepared for Vladimir Reshetnikov

21 September 2026

Scope. All 31 questions in research areas R1–R9 of `docs/proposals/11-further-improvements`, together with their engineering prerequisites, relevant Lean APIs, and the proposed acceptance gates.

Repository snapshot.

784664ff34e819422510a4d470ab07fe48bb736b

Lean toolchain: `leanprover/lean4:v4.34.0`.

Evidence boundary. This is a source-based investigation, with explicit mathematical arguments and eight executable finite-model checks. It is not a claim that the proposed engine changes have been implemented or that the repository’s Lean benchmarks have been rerun.

The accompanying archive contains this self-contained L^AT_EX source, its compiled PDF, a standard-library-only Python experiment, its results, and build instructions. Recommendations are distinguished from established results; negative answers state the exact bound or model to which they apply.

Abstract

The further-improvements proposal identifies the right bottlenecks: interacting metavariables, carrier invention, residual evaluation, indexed induction, recursive-call synthesis, proof scheduling, retrieval, ranking, and optional learned guidance. Several of its expert questions already have useful partial answers in existing systems. Smyth removes trace completeness and supports higher-order examples; Hoogle+ handles type classes through dictionaries; Canonical demonstrates indexed synthesis and predicate synthesis with a supplied generalized induction scheme; and Lean 4.34.0 already supplies a proof-producing call-by-value evaluator. Other questions need a narrower specification before a completeness or reproducibility claim is meaningful.

This report answers each of the 31 questions individually. Its central design is a whole-state search with dependency-aware components, immutable ranking objectives, typed residual observations, guarded caches, recursion capabilities, and a single audited acceptance boundary. It gives a constructive finite-carrier search, proves a sound observation-based state lower bound, explains the limits of motive enumeration, and states sufficient conditions for conservative pruning, independent-goal reuse, bounded synthesis completeness, and deterministic replay. It corrects several consequential overclaims: open-goal counting is not an admissible heuristic in general; incomplete observation rows are not states; finite parametric tests do not cover arbitrary lengths; and sorting results cannot undo deadline-dependent discovery. A staged implementation plan moves existing Lean facilities and retrieval earlier, while keeping custom dependent evaluation and unrestricted helper invention as separately testable research.

Contents

1	Executive conclusions and evidence	3
1.1	What should change first	3
1.2	Sources, versions, and what was actually checked	3
1.3	Coverage of the questions	3
2	Semantic boundaries and corrections	4
2.1	The synthesis problem and its result types	4
2.2	The invariant needed for safe pruning	5
2.3	Corrections that affect the development plan	5
3	R1: search with interacting metavariables	6
3.1	R1.1: alternatives to copying or sequentializing all goals	6
3.2	R1.2: an admissible cost-to-close heuristic	7
3.3	R1.3: just-in-time learning and ranking soundness	8
3.4	R1.4: wall-clock limits and machine-independent results	8
4	R2: carrier discovery and fold synthesis	9
4.1	R2.1: can finite examples determine a minimal carrier?	9
4.2	A sound state lower bound from partial observations	10
4.3	R2.2: polymorphic automata and the correct alphabet	11
4.4	R2.3: backwards fusion and synthesis of auxiliary functions	12
4.5	R2.4: complete motive enumeration for primitive recursion	12
4.6	R2.5: induction generalization, rippling, and lemma speculation	13
5	R3: residual and bidirectional evaluation	14
5.1	R3.1: higher-order and dependent live evaluation	14
5.2	R3.2: incremental normalization and rollback invalidation	14
5.3	R3.3: the cheapest conservative evaluator	15
5.4	R3.4: top-down angelic search and completeness	16

6	R4. Indexed families and motive synthesis	17
6.1	R4.1. Can motive enumeration be complete, terminating, and closed under generalization? .	17
6.2	R4.2. How should transports be represented and canonicalized?	17
6.3	R4.3. What transparency policy is supported by existing experience?	18
6.4	R4.4. Can Lean’s motive computation be called as a library?	19
7	R5. Recursive programs from incomplete examples	19
7.1	R5.1. Does the capability interpreter agree definitionally with its realization?	19
7.2	R5.2. Does angelic execution compose with top-down search?	20
7.3	R5.3. Is there a canonical small basis of recursion schemes with measured coverage?	21
8	R6. Contracts, induction, and proof debt	22
8.1	R6.1. How should value holes and proof holes be interleaved?	22
8.2	R6.2. Which properties admit induction, simplification, and arithmetic?	22
8.3	R6.3. Which Lean induction facilities are usable now?	23
8.4	R6.4. Is there a principled policy for scheduling proof debt?	24
9	R7. Component retrieval at library scale	24
9.1	R7.1. What abstraction works for dependent, universe-polymorphic signatures?	24
9.2	R7.2. Has type-guided abstraction refinement been combined with type classes?	25
9.3	R7.3. Can premise selection use data goals and examples, and what corpus is suitable? . . .	26
10	R8. Ranking and equivalence	26
10.1	R8.1. Is there a defensible formal objective for ranking inhabitants?	26
10.2	R8.2. Which equivalence is cheap and meaningful for dependent candidates?	27
11	R9. Learned proposals and auditability	28
11.1	R9.1. What accuracy do models achieve on carrier and helper proposals?	28
11.2	R9.2. What determinism and auditability are possible beyond recording proposals?	28
12	An integrated architecture and implementation plan	29
12.1	Six interfaces with explicit trust boundaries	29
12.2	A proposed control loop	30
12.3	A revised sequence of milestones	30
12.4	Engineering items that still need semantic care	31
13	Validation, experiments, and limits of the evidence	32
13.1	What was executed for this report	32
13.2	The required Lean regression and ablation suite	33
13.3	Claims that this investigation does not make	34
14	Conclusion	34
A	Source map and question coverage	34
	References	36

1 Executive conclusions and evidence

1.1 What should change first

The highest-value next step is not to replace every subsystem at once. It is to make the current engine’s semantic boundaries explicit and then exploit capabilities already present in its pinned Lean version. In particular, indexed induction is not wholly absent from Lean’s low-level API: the current LEANT2 search deliberately filters it out. Similarly, an equation-based, proof-producing evaluator need not start as a new interpreter: `cbv` and `decide_cbv` are present in Lean 4.34.0. These are concrete starting points, not predictions of benchmark success [1, 2, 3].

The proposed architectural direction is:

Preserve a small checked acceptance boundary; make search states persistent and self-describing; treat observations and proof obligations as typed constraints; allow heuristics to change priority, but not the meaning of acceptance or of a negative result.

Three different guarantees must remain separate. *Acceptance soundness* means that a returned program has the requested type and checked contract under the chosen axiom policy. *Search conservativity* means that pruning and memoization do not discard a permitted solution. *Reproducibility* means that a specified logical run can be repeated. A kernel check at the end protects the first guarantee, not automatically the other two.

1.2 Sources, versions, and what was actually checked

The reviewed LEANT2 snapshot is commit 784664ff34e819422510a4d470ab07fe48bb736b; its toolchain file specifies Lean 4.34.0. The investigation read the proposal’s TeX sections and inspected `Leant2/Search/Core.lean`, `Leant2/Native/Transaction.lean`, `Leant2/Engine.lean`, `Leant2/Accept/Gate.lean`, and `Leant2/Core/Types.lean`. Lean’s induction and call-by-value tactic sources were additionally checked at the v4.34.0 tag [1, 2, 3].

The proposal’s performance evidence belongs to its own reported experiment at commit 33cec8b, not to a new run performed for this report. For example, it reports complete scores on six historical acceptance harnesses, thirteen unscored Church stretch cases that remain unsuccessful, and a particular profile where residual evaluation accounted for 45% of measured wall time. These are valuable observations, but they do not establish that all future workloads are evaluation-bound, or that a new cost model will preserve the same deadline-bound candidate lists [1].

No Lean executable was available in the report-generation environment. The included Python checks were executed; no Lean patch, compiled Lean example, benchmark speedup, model accuracy, or external-expert consultation is claimed. Lean-shaped snippets below are design sketches unless explicitly identified as quoting an inspected API. The mathematical propositions are proved in prose, not machine-checked in this archive.

1.3 Coverage of the questions

Question identifiers below follow the order of the numbered expert questions in each research area, rather than the order of the proposal’s final expert table. The question descriptions are paraphrases; their full answers have stable *Ri.j* headings.

Area	Questions	Principal outcome
R1 Search	4	Dependency components; valid lower bounds; fixed objectives; logical-work replay.
R2 Carriers	5	Bounded constructive search; sound state lower bounds; parametricity conditions; fusion and motive proposals.
R3 Evaluation	4	Typed residuals; versioned demand caches; existing certified CBV baseline; conditional angelic reasoning.
R4 Dependent types	4	Finite motive grammars; explicit transports; staged transparency; existing induction APIs.
R5 Recursion	3	Capability realization theorem; top-down angelic search; measurable scheme portfolio.
R6 Contracts	4	Proof-debt dependencies; a sufficient inductive fragment; existing tactics; fair scheduling.
R7 Retrieval	3	Dependent hypergraph abstraction; dictionary passing already in Hoople+; leakage-controlled program benchmarks.
R8 Ranking	2	Declared preferences and MDL; certified equivalence versus observation-based grouping.
R9 Learned proposals	2	A task-specific accuracy study; bounded, replayable, non-authoritative proposal interface.

2 Semantic boundaries and corrections

2.1 The synthesis problem and its result types

Fix an environment E , a local context Γ , a target T , a contract $C : T \rightarrow \mathbf{Prop}$, a provider set $\mathcal{L}$, and a trust profile $\mathcal{A}$. An accepted answer consists of closed abstractions of

$$\Gamma \vdash p : T, \quad \Gamma \vdash \pi : C(p),$$

with all transitive axiom dependencies allowed by $\mathcal{A}$. Where there is no contract, the second component is trivial. An executable answer additionally requires successful realization through the supported compilation path. That last condition is not identical to kernel type correctness.

The inspected gate already performs synchronous kernel checking, rejects unresolved expression holes and escaped locals, generalizes universe metavariables jointly, and audits the transitive axiom closure. The engine’s acceptance callback constructs the contract proof type from the extracted program, so the contract association is not merely left to an unrelated proof argument. The standard profile permits `propext`, `Quot.sound`, and `Classical.choice`; the strict constructive profile permits none of these axioms [1]. Preserve this distinction when adding evaluators and proof tactics: “kernel checked” does not mean “axiom free.”

A result should distinguish at least the following statements:

1. A program and its contract proof were accepted.
2. A particular candidate, or every candidate actually visited, was refuted.
3. A precisely described finite grammar was exhausted.
4. A kernel-checked proof establishes impossibility for the actual target or contract.
5. Work or time ran out without one of the stronger conclusions.

The existing result algebra already recognizes most of these distinctions. In particular, a count of rejected candidates must never become a universal impossibility claim merely because all visited candidates failed.

2.2 The invariant needed for safe pruning

Let s be a partial search state. Write $\text{Comp}(s)$ for its permitted, well-typed completions, including remaining universe constraints, local-context requirements, and the declared grammar bounds.

Definition 2.1 (Conservative refutation). A pruning procedure is conservative if its return of *refuted* implies

$$\neg \exists \theta \in \text{Comp}(s), C(p_s \theta). \quad (1)$$

Its return of *unknown* asserts no impossibility.

An evaluator that returns a plausible Boolean on an open expression is not thereby a conservative refuter. It needs either a proof of the implication in (1), or a verified semantics whose soundness supplies that proof schematically. Differential tests can discover violations, but agreement on a large finite test set is not a proof of the implication.

Proposition 2.2 (Safe use of conditional observations). *Suppose $A_s(\theta)$ is a set of inferred observations satisfying $C(p_s \theta) \Rightarrow A_s(\theta)$ for every $\theta \in \text{Comp}(s)$. If the conjunction $A_s(\theta) \wedge C(p_s \theta)$ is impossible for all such θ , then pruning s is conservative.*

Proof. A completion satisfying the contract would satisfy the inferred observations by the first hypothesis, contradicting the second. Notice that observations need not hold for completions that violate the contract. $\square$

This simple distinction is essential for recursive-call oracles: an example specifying $f(a) = b$ constrains a recursive call of every *valid* completion, not of every syntactically legal completion.

2.3 Corrections that affect the development plan

Issue	Corrected statement and consequence
Open-goal heuristic	A single unification can assign several holes. Summing a positive cost over all open goals need not be a lower bound. Start with zero, or with an explicitly justified relaxed problem.
Deadline determinism	Sorting accepted candidates fixes their order, not which candidates were discovered. Use completed logical epochs and replay receipts.
Carrier minimality	Minimize a declared representation-and-program cost, not an unspecified “smallest type.” Do not allow the fold package to capture its major input.
Observation rows	Missing entries are unknown. Distinct incomplete rows need not denote distinct states. Use witnessed conflicts, plus transition constraints.
Reverse as a fold	Reverse has a right-fold implementation with carrier <code>List A</code> : append a singleton at each step. Failure of a small body grammar is not nonexistence of that carrier.
Parametric tests	A generic position test can characterize one length under a suitable parametricity theorem. Testing finitely many lengths does not characterize all lengths.
Trace completeness	Smyth was designed to remove this requirement, and its implementation supports higher-order examples [16].

Issue	Corrected statement and consequence
Indexed synthesis	Canonical gives positive indexed and generalized-predicate examples, though the generalized induction scheme is supplied in its example [9].
Type-class retrieval	Hoog+ already incorporates dictionary constraints and instance components; higher-kinded variables are a stated limitation [23].
Existing evaluator	Lean 4.34.0 includes <code>cbv</code> and <code>decide_cbv</code> . Benchmark these before building a replacement [3].
Transport examples	For naturals, $n + 0$ reduces to n ; $0 + n$ is the appropriate simple example of an equality requiring propositional reasoning [6].
Regression gates	A changed ordering need not preserve deadline-truncated candidate lists. Separate exhaustive bounded equivalence from deadline performance.

These corrections do not invalidate the proposal’s overall direction. They change which claims can be hard acceptance gates, which are experimental hypotheses, and which facilities should be reused rather than reimplemented.

3 R1: search with interacting metavariables

3.1 R1.1: alternatives to copying or sequentializing all goals

Answer. Use an AND/OR search over *dependency components*, while retaining whole-state snapshots as the correctness baseline. There is no general representation trick that makes shared existential variables independent. The useful alternative is to identify the portions that really are independent, share immutable structure, and represent a component’s answer as a guarded substitution rather than as an unqualified “goal solved” flag.

Aesop supplies a practical best-first search precedent; its probabilities are priority annotations, not guaranteed semantic costs [7, 8]. For LEANT2, the relevant unit is a store of constraints plus an obligation list. The current implementation snapshots `Meta.SavedState` for an alternative and commits only when the whole continuation succeeds. That is a sound structural baseline to preserve during refactoring [1].

What independence must include. Construct a graph whose vertices include obligations, expression metavariables, universe metavariables, delayed assignments, local declarations, instance obligations, and residual observations. Add dependency edges not just for holes in the displayed target, but also for holes occurring in local types and values, metavariable declaration contexts, delayed-assignment environments, and pending contracts. An operation that may assign a shared metavariable induces a write edge. Unknown effects conservatively connect components.

For example, a target with no visible metavariable can still be coupled to another goal through a local instance whose type contains one. Conversely, two goals may mention the same immutable constant without being coupled. Constant sharing is a dependency on the frozen environment, not a shared writable hole. Computing connected components after every node would be wasteful: maintain incremental summaries, and recompute a component only when an assignment or context extension touches its summary.

Proposition 3.1 (A sufficient frame condition). *Let two component solvers have read sets R_1, R_2 and write sets W_1, W_2 in a fixed global environment. Suppose each solver is deterministic relative*

to the store it reads; all relevant reads and writes are included; fresh names are renamed apart; and

$$W_1 \cap (R_2 \cup W_2) = \emptyset, \quad W_2 \cap (R_1 \cup W_1) = \emptyset.$$

Then executing either solver does not change the other’s applicability or result, apart from fresh-name renaming, and their resulting substitutions can be combined.

Proof. Every location read by the first solver has the same value before and after the second solver, and conversely. Their writes are disjoint. Thus each run takes the same steps and produces the same assignments as it would in isolation. The union of the assignments is well-defined. The hypothesis that the read sets include typing and scope dependencies makes the same argument apply to the well-typedness of these assignments. $\square$

The proposition is a semantic contract for an implementation, not a claim that surface free-variable collection already satisfies it.

Memoization. A memo entry should contain a normalized goal-and-context fingerprint, the relevant environment and transparency settings, a dependency fingerprint, a solution expression, and any assignments it requires. On reuse, freshen private metavariables, instantiate the context substitution, validate the guard, and recheck the candidate. A memo holding one successful inhabitant does not justify throwing away all other inhabitants when a later contract can distinguish them. Cache a stream or family of alternatives, or prove that the component is observationally irrelevant to all remaining obligations.

Initially memoize only closed, immutable-context subproblems. Then add guarded open-context entries. This earns useful reuse before implementing a complete dependency-aware AND/OR engine. Snapshot costs in the historical profile were small; eliminating copying is not itself evidence that the total search becomes faster [1].

3.2 R1.2: an admissible cost-to-close heuristic

Answer. There is no generally valid positive “one unit per open goal” rule. Admissibility is relative to a fixed cost model. It requires

$$h(s) \leq \min_{p \in \text{Comp}(s), C(p)} \text{remaining cost of producing } p.$$

The cost of a search transition and the final ranking cost of a program are also different quantities unless explicitly aligned.

A single application may unify a target $R?x?y$ with Rt_1t_2 and assign both argument holes while solving a proof hole. Charging separately for each assigned hole can double-count the same transition. Exact-local steps can even have zero cost in the proposed cost table. The supplied finite-model check shows the basic obstruction with two obligations closed by one unit-cost action: the sum heuristic is 2 while the true residual cost is 1.

The safest initial heuristic is $h = 0$. A stronger one comes from an *optimistic abstract problem*: erase constraints, allow some assignments for free, and solve the easier problem. To transfer its lower bound, every concrete completion must induce an abstract completion of no greater cost. Abstract hypergraph distances can then be used, but only after verifying this simulation property for every search rule.

Combining bounds. For coupled goals, the maximum of individually valid lower bounds remains a lower bound. Their sum need not. Summation is justified only for components that require disjoint charged work, or by an explicit cost partition assigning each concrete transition’s cost among abstract subproblems without exceeding that cost. Mere lack of shared metavariables is insufficient if a single charged provider action is counted in several components.

For a true best-first minimum-cost claim, keep the incumbent’s objective $J(p)$ and lower bounds $L(s)$ on all unfinished branches. Certification of optimality requires

$$J(p) \leq \inf_{s \text{ on frontier}} L(s).$$

Changing priorities with a learned score does not invalidate this criterion, but stopping before it holds yields only “best found.” Zero-cost transitions also need a finite closure or cycle control; otherwise a zero-cost layer can prevent all positive-cost work from being reached.

3.3 R1.3: just-in-time learning and ranking soundness

Answer. Learning from partial successes is a reasonable search heuristic, but it is not a semantic justification for ranking. Probe provides an explicit just-in-time learning precedent for bottom-up enumeration [10]. Transferring its mechanism to dependent top-down search is an experiment: a rule opens a state-dependent subtree, and the distribution of useful rules changes as holes and examples are resolved.

Separate three fields in the design:

$J(p)$ final objective, $L(s)$ valid lower bound, $P_e(s)$ priority in learning epoch e .

Only P_e should change when the learner updates. Freeze J for a query, or version it visibly if the user changes preferences. Retain L independently of the learned predictions. A learner that prefers a subtree cannot establish that the subtree contains the intended program, or that a disfavored subtree has no better solution.

An implementable control is epoch-based adaptation: collect observations during a fixed logical batch; aggregate them in a canonical order; update integer or rational scores; freeze those scores for the next batch. Record the update data and a model-version digest. Reserve a nonzero deterministic share for a complete bounded baseline enumerator. This separates reproducible learning from scheduler-dependent races and prevents permanent starvation.

The stopping receipt should say either “best found under objective version v ” or “minimum certified over grammar G_B .” A deadline-limited search is allowed to return different valid programs under different priorities. What would be misleading is to call those differences semantically optimal, or to hide the changed objective from a ranking comparison.

3.4 R1.4: wall-clock limits and machine-independent results

Answer. In general, a fixed positive wall-clock deadline, unrestricted machine speeds, and a result that uses all work completed by that deadline cannot all coexist with identical candidate sets.

Proposition 3.2 (Deadline obstruction). *Suppose a deterministic run first discovers a new accepted candidate after positive work w . On two machines executing the same logical run at rates v_1 and v_2 , a common deadline D can fall between w/v_2 and w/v_1 . A policy returning all candidates discovered before the deadline then produces different candidate sets.*

Proof. Choose rates with $v_1 D < w \leq v_2 D$. Only the faster run reaches the discovery. Sorting either set cannot insert the missing candidate in the slower set. $\square$

A calibrated node budget is not a complete cure: different nodes may have very different normalization, unification, or proof costs. Startup calibration also makes the chosen logical budget machine-dependent.

Use two services instead. An *interactive service* respects wall time and reports its completed logical prefix. A *replay service* runs the recorded prefix, independently of how long it takes. Deterministic checkpoints can expose only completed epochs; the number of completed epochs may differ by machine, but a named epoch has a reproducible meaning. A fixed logical budget gives identical results under deterministic execution when both machines finish it; a hard earlier deadline can only return an explicitly incomplete run.

Parallel workers should commit completed batches in canonical order, rather than cancel every competitor on the first result and hope that sorting fixes the race. Use deterministic tie-breaks, stable provider order, fixed random seeds, and recorded external proposals. General restart schedules can be useful heuristics, but the classical Luby theorem for randomized Las Vegas restarts is not automatically a theorem about a portfolio of correlated deterministic Lean tactics [11].

4 R2: carrier discovery and fold synthesis

4.1 R2.1: can finite examples determine a minimal carrier?

Answer. Yes for a fully specified finite search problem; no unique semantic “minimal Lean type” follows from finite examples alone. The object to optimize is a typed package, not just the spelling of its carrier:

$$(K, z, s, q), \quad z : K, \quad s : A \rightarrow K \rightarrow K, \quad q : K \rightarrow B, \quad h(xs) = q(\text{foldr } s \ z \ xs). \quad (2)$$

Its cost should include the carrier, seed, step, finalizer, required helpers, and proof obligations. Type-expression size, state cardinality, runtime storage, and source-code size are different objectives. Isomorphic types can have quite different library support and elaboration costs.

A scope restriction is essential. The package may depend on fixed parameters, such as a mapping function, but must not capture the major input list xs outside its fold. Otherwise take $K = 1$, ignore the fold, and let q compute the whole answer from the captured xs . Even without that capture, allowing an unrestricted finalizer merely moves the original task into q : a list-valued carrier can store the entire input and an arbitrary finalizer can process it later. That is a valid representation, but not necessarily progress in synthesis. Bound and charge the entire package, and disallow calls to the unfinished target except through approved recursion capabilities.

A constructive finite procedure. Choose finite provider and literal sets, a finite carrier grammar at size bound b_K , finite syntax bounds for z, s, q , and a fixed universe/context policy. For a finite DSL with decidable typing and exact terminating evaluation, the following procedure decides sample consistency at those bounds:

```
for package in all_packages_in_nondecreasing_fixed_cost(bounds):
  if not well_typed(package): continue
  if every_sample_matches(package):
    return MinimumSampleConsistent(package, bounds)
return Exhausted(bounds)
```

Every syntactic candidate is visited; every check terminates; the first accepted cost is minimal. These are the three facts needed for the guarantee. In a Lean implementation, a bounded tactic or normalization attempt that times out must produce *unknown*, not a false typing or evaluation verdict. The engine may then claim exhaustion only of the explicitly bounded operational search, not of all semantically valid packages with the same type-size bound.

For finite alphabets and finite carriers, an even cleaner exact subproblem is to enumerate total tables. With alphabet size a , state count n , and output set size b , there are

$$n^{an} \cdot n \cdot b^n$$

labelled choices of transition function, initial state, and finalizer. Evaluate each table on the samples. An implementation may use SAT or a finite-domain solver, but exhaustive enumeration already supplies a constructive decision procedure. Reifying a satisfying table into a permitted Lean carrier is a separate checked step; a table with n states does not prove that an arbitrary carrier grammar has an admissible representation of those states.

The included experiment exhausts all tables with one and two states for binary parity on words of length at most four. It finds no one-state solution and two labelled two-state solutions. A second experiment on unary length modulo three finds none with one or two states and six labelled three-state solutions. These are exact finite results, not evidence that the full carrier-invention problem has become easy.

What finite examples cannot establish. A package consistent with the samples may fail on longer inputs. A minimal sample-consistent machine need not be the minimal machine for the intended function. Nor does the failure of one bounded body grammar prove that its carrier is insufficient. “No solution” is meaningful only after specifying which of these finite or universal problems was actually decided.

4.2 A sound state lower bound from partial observations

There is a useful replacement for counting distinct incomplete observation rows. Fix a finite set V of suffixes. Join $x, y \in V$ by an edge when a prefix u has been observed with both $h(u ++ x)$ and $h(u ++ y)$ defined in the sample table, and these outputs differ. Call the resulting graph G the *witnessed incompatibility graph*.

Theorem 4.1 (Observed state lower bound). *For a uniform fold package (2) satisfying the observations, the map $x \mapsto \text{foldr } s \ z \ x$ assigns different carrier states to adjacent vertices of G . Consequently, if K has n states, then*

$$\chi(G) \leq n.$$

In particular, a clique of size r certifies that fewer than r carrier states cannot suffice for the observed behavior.

Proof. Suppose suffixes x, y produce the same state k . For every prefix u , the fold identity

$$\text{foldr } s \ z \ (u ++ x) = \text{foldr } s \ (\text{foldr } s \ z \ x) \ u$$

shows that both concatenations produce $\text{foldr } s \ k \ u$. Applying the same finalizer gives equal outputs. Therefore a prefix witnessing unequal outputs prevents equality of the two suffix states. The state assignment is a proper coloring with at most n colors. $\square$

This proof uses only actual conflicting observations and fold composition. It does not assume a completed observation table, a membership oracle, or a particular grammar for the step.

For example, the three partial rows

$$(0, ?), \quad (?, 0), \quad (1, ?)$$

are different as strings, but only the first and third are incompatible. Two states can explain the table; counting three distinct rows as three required states is unsound. Compatibility of partial rows is not transitive. Coloring is a necessary condition, not sufficient for a transition-consistent machine: transition congruence imposes additional constraints.

A cheap implementation can use a found clique rather than compute the exact chromatic number. A hard prune against a proposed finite carrier should include a checked cardinality bound and the conflicting sample witnesses. Counting values *constructible by a small expression grammar* is not a substitute for counting all states the fold can reach, unless reachability inside that bounded grammar is itself the stated model.

For comparison, full behavior defines the left-context equivalence

$$x \sim_h y \iff \forall u, h(u ++ x) = h(u ++ y).$$

Its quotient has well-defined prepend transitions and an output map. This is a semantic minimal-state construction on reachable input states, but the quotient can be infinite and need not have a decidable equality or a useful constructive representation in the chosen grammar. Classical fold characterizations and constructive refinements are directly relevant background: Weber and Caldwell explicitly give evidence-dependent constructive transformations in Nuprl, not an oracle extracting an implementation from arbitrary finite samples [12, 13].

4.3 R2.2: polymorphic automata and the correct alphabet

Answer. Separate a genuinely parametric fragment from operations that inspect or compare elements. For a pure relationally parametric function $h : \forall A, \text{List}(A) \rightarrow \text{List}(A)$, the useful finite observation at input length n is its action on the list of distinct positions $[0, \dots, n-1]$ of type $\text{List}(\text{Fin}(n))$.

Assume the naturality property

$$\text{map}(f)(h_A(xs)) = h_B(\text{map}(f)(xs)) \quad (f : A \rightarrow B). \quad (3)$$

Every length- n list xs is the image of the position list under the function sending a position to its element. Thus the output at that generic position list determines the output at every list of that length, including lists with repeated elements. The argument also constrains whether the function duplicates, drops, or reorders positions. It does not identify its behavior at every other length.

For any fixed N , the identity and the function that returns its input when its length is at most N and returns the empty list otherwise are both parametric and agree on all tests up to length N . They disagree at $N+1$. The finite-model experiment explicitly checks this at $N=4$. Therefore “a single monomorphic instance suffices” must not be confused with “finitely many examples suffice.” The polymorphic-testing literature addresses the former issue under parametricity hypotheses [14].

With an explicit equality dictionary, position identity alone is not enough to model all observations: equality patterns of inputs matter, and the dictionary must satisfy the laws used by the evaluator. For arbitrary Lean definitions, polymorphic syntax alone is not a proof of (3). Classical type inspection, unrestricted providers, or extra structure can invalidate the intended free theorem. Require an audited syntactic fragment with a relational interpretation, or a separately proved parametricity theorem for the components.

Recommendation. Start with generic-position observations as a *proposal and testing* mechanism for a restricted list DSL. Use finite equality-pattern or register-style abstractions only when the supported equality operations are explicit. Treat the extension to arbitrary dependent Lean types and instances as a new semantics problem, not as a change of alphabet in an ordinary finite automaton. Angluin-style exact learning also assumes access to specified query oracles; a user’s fixed sample set does not provide an equivalence oracle [15].

4.4 R2.3: backwards fusion and synthesis of auxiliary functions

Answer. Backwards fusion is useful when it generates *checkable local algebra equations*. It is not a general complete carrier-invention procedure. Suppose $H = \text{foldr } s \ z$ and seek $h = q \circ H$. For a known recursive specification of h , try to solve equations relating $q(s(a, k))$ to the specification’s constructor case, together with $q(z)$ for the base case. A successful solution must cover every reachable k , or a proved invariant that contains all reachable states; finite examples alone do not establish this.

Reverse illustrates the distinction. A perfectly valid choice is

$$K = \text{List}(A), \quad z = [], \quad s(a, k) = k ++ [a], \quad q(k) = k.$$

Induction on the input proves that the fold computes reversal. For the endofunction representation one may instead take

$$K = \text{List}(A) \rightarrow \text{List}(A), \quad z = \text{id}, \quad s(a, k) = \lambda r. k(a :: r), \quad q(k) = k([]).$$

The stronger invariant $H(xs)(r) = \text{reverse}(xs) ++ r$ proves correctness. The first representation repeatedly appends to a growing list and has quadratic list-traversal work under the usual linked-list cost model. The second exposes accumulator-style computation. This is an efficiency and grammar issue, not a proof that `List A` cannot be a carrier.

Useful completeness boundaries. For fixed finite algebras, fixed expression bounds, and exact local semantics, exhaustively solving these equations is complete within the stated finite space. For a list homomorphism, a fixed candidate binary operation and its identity can be checked against the required algebraic laws. For polynomial functors, a fixed carrier and finite collection of constructor operations lead to analogous finite local obligations. Inventing the carrier, an invariant, and arbitrarily complex operations is a different, unbounded problem.

A practical search should use failed equations as hints. When q cannot recover information needed by a constructor case, propose a product carrier containing an additional statistic. When a result must be extended with an unknown tail, propose a function-valued carrier. When two recurrences repeatedly recompute related quantities, propose a tuple. Keep the former representation as an alternative; a failed heuristic equation search is not a semantic refutation. The constructive characterization literature is a source of proof obligations and transformations, while the finite grammar is what gives the implementation its actual decision boundary [12, 13].

4.5 R2.4: complete motive enumeration for primitive recursion

Answer. There is a simple complete *schema* for a declared nondependent primitive-recursive fragment with fixed parameter types. It is not a small complete search procedure for all dependent primitive recursion. For

$$f : \mathbb{N} \rightarrow S_1 \rightarrow \cdots \rightarrow S_k \rightarrow T,$$

use the constant recursor result type $K = S_1 \rightarrow \dots \rightarrow S_k \rightarrow T$. The successor branch receives an induction result of type K , so recursive calls at the predecessor may use newly computed parameter values. If result tuples or further accumulators are allowed, enlarge K by the corresponding product and function constructors.

Completeness for this fragment is conditional on enumerating all its base and step expressions. With finite constructor/provider/literal sets and a size bound, that is a finite grammar argument. Across increasing bounds, fair enumeration is a semidecision procedure for expressible solutions, not a terminating decision procedure for every failure. A cap such as carrier size three is an engineering bound, not a completeness theorem for primitive recursion.

Dependent motives $P(n, x)$ require a different grammar because the types of parameters and results may change with n . Use a stratified sequence: constant result, generalized fixed parameters, products, index abstraction, then invariant-bearing dependent families. A finite target-occurrence grammar is not closed under adding an accumulator: adding a new argument changes the binding structure, not just an occurrence of an old index.

To control natural-number explosion, charge introduction of recursive structure, not just the number of recursive calls exposed by a recursor. Use observed numeric constants and a small literal basis; prefer already available arithmetic providers; and bound nested eliminators separately from term size. Retain fair later access to excluded numerals or schemes when claiming semicompleteness. None of these controls establishes a universal polynomial-time search bound.

4.6 R2.5: induction generalization, rippling, and lemma speculation

Answer. The transferable idea is to use *failed proof alignment* as evidence about a missing parameter or invariant. Do not transfer every heuristic as a hard pruning rule. A branch where the induction hypothesis speaks about $g(xs, a)$ but the recursive call requires $g(xs, a')$ suggests generalizing a . A goal comparing a recursive result followed by an append with an unextended hypothesis suggests an accumulator invariant. Repeatedly failing to relate two computed statistics suggests tupling or an auxiliary lemma.

For LEANT2, implement a small proposal generator: compare a stuck branch goal with available induction hypotheses, anti-unify their rigid structure, and identify arguments whose differences are accounted for by a missing binder. Compute the transitive dependency closure of any variable to generalize, then attempt the new induction in an isolated snapshot. A proposed lemma receives its own typed proof obligation and resource budget. It becomes reusable only after kernel checking and axiom auditing.

Canonical offers an especially relevant partial precedent: its paper’s reverse-involution example supplies a generalized list-induction rule and the solver synthesizes the needed predicate. That demonstrates that predicate instantiation can be automated once the rule is available; it does not show that the system automatically discovers every required rule [9]. This suggests a controlled first experiment: give both the current engine and the improved engine the same small library of generalized schemes, measure predicate discovery separately, then measure scheme proposal as a second task. Otherwise failures of scheme selection, motive inference, and body search are confounded.

5 R3: residual and bidirectional evaluation

5.1 R3.1: higher-order and dependent live evaluation

Answer. Higher-order live evaluation is not an untouched starting point. Smyth’s core exposition uses first-order function examples for simplicity, but its implementation supports higher-order function examples. Its hole closures and resumption rules are directly relevant, as is the later Scribe work on example propagation [16, 17]. The genuinely difficult extension for LEANT2 is dependent typing across changing hole assignments and contexts.

Represent a suspended occurrence as a *typed closure*, not a bare hole identifier:

$$(m, \Delta, \rho, T, \nu),$$

where m is declared in telescope Δ , ρ is a well-typed substitution from that telescope into the evaluation context, T is the expected result family after substitution, and ν records the relevant state version. Two occurrences of one hole under different environments are related constraints on the same unknown term, not the same value.

For a function hole g , an observation can say that applying it to argument a must produce b . Multiple observations for g must be simultaneously realizable by one function. For a dependent function $g : (x : A) \rightarrow B(x)$, the outputs live in different fibers $B(a)$. A table of untyped values loses exactly the information needed to justify substitution, comparison, and transport. One must preserve the expected fiber or use a heterogeneous relation with explicit equality evidence.

The first implementation obstacle. A delayed metavariable assignment can mention a hole whose local telescope has been changed by abstraction or recursion. The current `instantiatePartial` explicitly documents that it can build expressions whose remaining metavariable contexts no longer match, and restricts their use to reduction rather than assignment [1]. This is not by itself a demonstrated bug. It does mean that a proof of conservative residual evaluation cannot simply appeal to a well-typed substitution lemma without first proving that the partial-instantiation representation satisfies its premises.

Start with a typed first-order inductive fragment; add finite higher-order application observations; only then add dependent families and transports. When the supported fragment ends, retain a neutral expression and return *unknown*. Falling back must not silently reinterpret a dependent value as a value in the wrong fiber.

5.2 R3.2: incremental normalization and rollback invalidation

Answer. The right general precedent is dependency-tracked, self-adjusting computation, not an assumption that a cache of normal forms remains valid across rollback. Acar, Blume, and Donham give a correctness and consistency semantics combining memoization and change propagation [18]. That supplies a conceptual model; it is not a ready-made Lean metavariable normalizer with the required invariants already proved.

A residual evaluation node should record its expression or closure key, the assignment and environment facts it actually read, its resulting value or neutral residual, and edges to dependent nodes. On an assignment, invalidate or resume only affected nodes. On rollback, restore the persistent graph version or reject entries whose read fingerprints no longer match. Invalidating only when a hole becomes assigned is insufficient: the same hole name can occur in another branch with a different assignment, context, or delayed environment.

Proposition 5.1 (Guarded evaluation reuse). *Suppose an evaluator is deterministic and every semantic dependency of a cached result is included in its guard. If the expression, context interpretation, and every guarded dependency have the same meaning in a new state, reusing the result is equivalent to reevaluating it there.*

Proof. The evaluator sees the same expression and the same values for every inspected input. Determinism therefore gives the same result. An implementation that records too few dependencies does not meet the hypothesis. $\square$

The guard includes more than assigned expression holes: universe assignments, local declaration types and values, instance choices, equation-lemma sets, transparency, options affecting reduction, and the environment fingerprint may matter. Hashes are indices for these data, not logical equality proofs; verify structural equality on collisions where correctness depends on it.

The current `SearchCtx` stores `residualBlockers` in an `IO.Ref` outside `Meta.SavedState`; its checks examine whether remembered holes are still present or assigned. That is an optimization to scrutinize when making states first-class. A residual cache should be owned by its root program and state version, whereas the monotone work ledger should deliberately remain outside rollback. These two kinds of mutable state have different lifetimes [1].

The included rollback model demonstrates the failure mode, not a reproduced LEANT2 defect: caching value zero under one hole identifier and reusing it after the same identifier denotes value one produces a stale answer. A branch-and-assignment-aware key avoids that example. A production guard must cover the complete dependencies, not merely mimic that toy key.

5.3 R3.3: the cheapest conservative evaluator

Answer. First benchmark Lean’s existing proof-producing call-by-value machinery. The pinned version includes `Lean.Elab.Tactic.Cbv`, whose `evalDecideCbv` applies `of_decide_eq_true` and calls the CBV goal checker. The wrapper exposes `cbvGoal`, `cbvHyp`, and `cbvDecideGoal` from the meta-level implementation [3]. This makes a concrete fourth baseline alongside the current kernel reduction, ordinary simplification, and a prospective custom evaluator.

The current reference manual describes CBV as equation-based propositional rewriting, including support for well-founded definitions. It does not add code-generator trust as `native_decide` does, but its standard-axiom use still matters to a strict constructive profile. It also documents a limitation on rewriting dependent positions [4]. Therefore it is a strong candidate for closed or suitably nondependent observations, not a complete solution to typed open-hole resumption.

A low-risk implementation ladder. First split contracts into independently reportable observations using proved equivalences. Syntactic conjunction splitting is already implemented; extending it to Boolean conjunction, a concrete list’s `all`, and decidable equality requires the corresponding reflection lemmas. In particular, an arbitrary `BEq` instance does not justify replacing `x == y` by `x = y` without lawfulness evidence.

Second, apply ordinary proof-producing simplification or CBV to each observation, with cached expression sharing and explicit resource limits. A failed or stuck call is unknown. Third, introduce a small verified evaluator for the hottest supported fragment. Its semantic theorem should state that every reported concrete result agrees with the denotation of the quoted well-typed expression, and every reported refutation excludes all permitted completions. Such a fragment may exclude dependent eliminators while still improving many examples.

A fast untrusted interpreter can also propose a refutation certificate. Only a successfully

checked certificate causes a hard prune. If certificate generation or checking fails, keep the branch. When proof production is too costly for every node, the engineering choice is between a verified abstract transformer and a heuristic signal that changes priority only. There is no third option in which an unverified false result becomes safe merely because surviving candidates later pass the gate.

What Lean4Lean contributes. The current Lean4Lean paper reports a complete alternative typechecker and correctness proofs for parts of the kernel, not a fully verified, automatically extractable open-hole evaluator for arbitrary dependent synthesis [19]. Reusing its definitions or proved lemmas can reduce duplicated metatheory. Porting rules alone does not transfer a theorem about different closure representations, partial substitutions, or the host evaluator. State the simulation relation first; then identify which existing lemmas actually prove its cases.

5.4 R3.4: top-down angelic search and completeness

Answer. Top-down angelic reasoning is feasible, but Burst’s theorem does not transfer automatically. Burst’s completeness statement assumes a complete angelic synthesizer and valid explanations of failure. Its outer algorithm also checks candidates under ordinary semantics and backtracks over strengthenings [20]. A top-down implementation must establish the same kinds of conditions for its own grammar and constraint representation.

For a recursive call at a known argument, an angelic semantics permits an output consistent with the current contract information. Equal calls of one function must agree when their arguments are known equal; dependent calls must return values in the correct fibers. Ignoring these restrictions is an overapproximation and may merely waste search. Enforcing an unjustified equality or restricting an unknown output too narrowly can instead remove a real solution.

Use the following contract:

Every ordinary execution of every valid completion has a corresponding angelic execution admitted by the residual constraint store.

Unsatisfiability of that store is then a conservative pruning reason. A satisfying angelic store is only permission to continue: its separately chosen outputs may not arise from a single actual recursive program. Reevaluate the completed body under ordinary semantics and subsequently pass the full acceptance gate.

Proposition 5.2 (Bounded top-down completeness). *Fix a finite typed candidate grammar, exact terminating sample evaluation, and a search whose branch pruning is conservative. Suppose the search enumerates a finite set of bounded derivations, visits every unpruned candidate, terminates after that enumeration, and no refinement permanently discards a sample-correct candidate. Then search terminates and finds a sample-correct candidate whenever one exists in that grammar; otherwise it exhausts the grammar.*

Proof. There are finitely many candidates. Each evaluation terminates. Conservativity and refinement preservation leave every sample-correct candidate available, and fairness eventually visits it. If none exists, exhausting the finite set terminates with failure at the declared bounds. $\square$

The assumptions contain the real work. An unbounded value domain, an infinite refinement sequence, or a fuel-limited evaluation returning unknown does not satisfy the exact finite premises. Fuel exhaustion is not a refutation. A practical bounded system can ensure progress by blocking an already disproved candidate, retaining unresolved alternatives, and recording which operational bounds prevented a complete verdict.

6 R4. Indexed families and motive synthesis

6.1 R4.1. Can motive enumeration be complete, terminating, and closed under generalization?

Answer. Yes for a precisely bounded syntactic class, but the class of occurrence abstractions alone is not closed under accumulator invention. Three different operations should be separated: abstracting indices already in a target; reverting existing parameters into the induction hypothesis; and inventing a new carrier or invariant not already present in the context. The first is largely elaborator engineering. The second is a small but important search. The third contains the genuine synthesis problem.

For example, mapping a length-indexed vector needs the motive

$$M(n, v) = \text{Vec } B \, n.$$

An accumulating traversal may instead need

$$M(n, v) = \prod_{a:S(n,v)} T(n, v, a),$$

or an invariant-carrying result. Selecting some occurrences of n in the first expression cannot invent the new binder a or its type. Consequently, even a complete search through maximal-occurrence abstractions is incomplete for the second class.

A useful finite specification is a *motive template language*. Its terminals are scoped subexpressions of the target, local types, and an explicitly bounded set of type constructors. Its operations are dependency-closed abstraction, reversion of a bounded number of locals, and bounded products, arrows, and subtypes. Bound expression size, universe-expression size, and the number of new binders. Enumerate the resulting finite syntax, discard ill-scoped terms, and check each remaining candidate in the appropriate telescope. If checking and the permitted constraint solving terminate exactly on this fragment, the procedure is complete and terminating for that grammar. Operationally timed-out checks must remain unknown rather than becoming evidence of nonexistence.

Dependency closure is essential. Abstracting an index changes the types of later variables that mention it. Those variables must be transported or reabstracted as well. One cannot independently replace two textually identical occurrences when their binding or type dependencies differ. Nor should “maximal occurrence” mean “largest printed substring”: the operation is on scoped expressions and their dependency graph.

Recommendation. Start with one inferred motive for a local whose indices are distinct variables, then a short ordered list of explicit generalizations. Keep genuinely invented motives in the same bounded carrier/invariant service used by R2 and R6. This gives a clear progression from engineering to research and avoids charging a basic vector-map example for an unrestricted higher-order search. The existing Canonical examples also show that indexed synthesis is not an entirely unexplored capability, although they do not establish completeness for arbitrary motives [9].

6.2 R4.2. How should transports be represented and canonicalized?

Answer. Use typed transport nodes with checked equality evidence; normalize a selected index language, not all propositional equality. There is no general cheap canonical representative modulo

arbitrary propositions that Lean may prove. Replacing this problem by heterogeneous equality or a setoid changes its presentation, not its underlying proof obligations.

If $P : I \rightarrow \text{Type}$, $x : P(i)$, and $e : i = j$, the transport has type $P(j)$:

$$\text{cast}(\text{congrArg } P \ e, x) : P(j).$$

The indices i, j and family P must remain in the internal representation. Proof irrelevance can identify two proofs of the *same* equality proposition; it does not identify arbitrary endpoints or make all transports disappear computationally. A display layer may hide a checked cast, but the term submitted to the kernel must still contain the required transport unless conversion makes it unnecessary.

A practical normalization discipline is to select a terminating, deterministic rewrite system for an index fragment and produce an equality certificate with every rewrite. Normalize each endpoint using that policy, compose the certificates, and reuse one transport construction for that normalized pair. Unknown index equations remain proof obligations. This yields canonicalization *relative to a policy*, not modulo all provable equalities. The policy and its lemma set belong in cache keys and receipts.

The proposal's example should be reversed: for Lean's natural-number addition, $n + 0$ is definitionally equal to n , whereas $0 + n = n$ generally needs a proof when n is a variable [6]. Therefore `Vec A (n + 0)` is not a good test of propositional transport. Use, for example, a vector at index $0 + n$ or an associative rearrangement of an index expression.

Heterogeneous equality is useful for stating that values in different fibers correspond. It is not a universal congruence engine. Even the current Lean `cbv` documentation explicitly identifies restrictions on rewriting in dependent positions; changing to heterogeneous equality does not automatically supply the missing dependent congruence principles [4]. For the first implementation, restrict transport to a registered family and a small index theory. Charge proof and normalization work separately from the presentation-size score, rather than calling every cast literally free.

6.3 R4.3. What transparency policy is supported by existing experience?

Answer. The inspected systems support selective normalization, not indiscriminate full unfolding. Canonical describes unfolding aliases of function types, reducible definitions, and type-class instances in its translation. The `sauto` interface distinguishes heuristic unfolding from explicitly forced unfolding, with special treatment of basic logical connectives [9, 21]. Neither description establishes one globally optimal transparency policy for LEANT2.

Use three explicit levels. At the first level, compute weak-head forms needed to recognize binders, inductive heads, and obvious reducible aliases. At the second, unfold a specific definition whose head is blocking an application, an index comparison, or motive construction. At the third, permit a larger but bounded normalization attempt, with a separate work charge and cache namespace.

A failed first-level comparison is not proof that two types differ. It is a request either to postpone a constraint or to try the next permitted level. Conversely, normalizing every definition before retrieval can erase useful library boundaries, duplicate large expressions, and spend work on arguments whose values are irrelevant to the current goal. Preserve high-level function names for retrieval and presentation even when a checked reduced form is cached for conversion.

The simplest experiment compares fixed reducible-head normalization against this three-level policy on identical logical search traces. Record the number and cost of successful escalations, expression growth, cache hit rate, and candidates lost only to resource limits. Do not choose a policy from one profile whose runtime was dominated by residual contracts rather than conversion.

6.4 R4.4. Can Lean’s motive computation be called as a library?

Answer. Yes. The pinned Lean 4.34.0 source exposes relevant operations in `Lean/Meta/Tactic/Induction.lean` [2]. In particular:

Lean.Meta.getMajorTypeIndices retrieves indices subject to the low-level induction routine’s restrictions, including variable-form indices and checks on repeated or improperly dependent occurrences.

Lean.Meta.mkRecursorAppPrefix constructs a recursor application through its motive, abstracting the major premise and indices from the target.

Lean.MVarId.induction orchestrates reversion, reintroduction, motive construction, and creation of the constructor subgoals. Returned `InductionSubgoal` records include the new goal, fields, and a local-variable substitution.

These operations compute a motive for the prepared target; they do not discover an arbitrary stronger invariant. Their availability answers the library-access question, but not the empirical question of whether repeated calls are cheap enough on a particular workload.

The current LEANT2 rule calls `MVarId.induction` after requiring zero indices and excluding `Nat`. Those exclusions are in the search rule, not a statement that Lean lacks the corresponding induction facilities [1]. A small first experiment should admit an indexed family only when its indices satisfy the low-level preconditions and its recursor is supported. For a non-variable index such as $n + 1$, use a prepared generalization with a remembered equality or an elaborator-level induction path that handles the expression; do not pass an incompatible major premise to the low-level routine and interpret the resulting exception as logical impossibility.

Wrap every attempted preparation in a transaction. Classify failure as unsupported index shape, prohibited elimination from `Prop`, failed type constraint, unsupported recursor, or exhausted resources. Preserve these categories in traces. Cheap syntactic prechecks can avoid predictable failures, but no particular millisecond bound is established by this report. The wrapper should also expose the resulting dependency substitution to the search rather than reconstructing it from printed variable names.

7 R5. Recursive programs from incomplete examples

7.1 R5.1. Does the capability interpreter agree definitionally with its realization?

Answer. Not in general. A structural realization can have useful definitional computation rules, but well-founded recursive equations are often available only as propositional equalities. Lean’s documentation explicitly distinguishes the two cases [5]. The correct interface should promise a proved simulation property, not universal definitional equality.

For a recursive state $x : S$, a relation $R : S \rightarrow S \rightarrow \text{Prop}$, and result family $P : S \rightarrow \text{Type}$, the fully dependent capability is

$$\text{rec}_x : \prod_{y:S} R(y, x) \rightarrow P(y).$$

Include every changing parameter in the recursive state or in the family over which the induction hypothesis is generalized. A capability that mentions only the original major argument is insufficient when recursive calls change an accumulator, an additional list, or a dependent result index.

Suppose a typed body language has an interpreter

$$\text{eval}(k, b, x) : \text{Option}(P(x)),$$

where k is fuel, and a realization $\text{realize}(b)$ obtained from a checked well-founded recursion construction. The desired one-sided theorem is

$$\text{eval}(k, b, x) = \text{some}(v) \implies \text{realize}(b)(x) = v. \quad (4)$$

Its assumptions must include well-typed bodies, valid decreasing-call certificates, sound interpretation of every primitive, and the equations used by the realization. Prove it by induction on the successful evaluation or by well-founded induction, using the realization’s equation theorem at each call. The interpreter’s **none** is an inconclusive resource result, not the value of the realized function and not evidence that a contract is false.

This interface is deliberately weaker, and more useful, than claiming the interpreter computes exactly what `Kernel.whnf` does on a raw `WellFounded.fix` term. Once an interpreted observation is checked against (4), it can support a theorem about the realized function even when conversion alone cannot perform the same computation.

Recommendation. Benchmark existing equation-based `cbv/decide_cbv` before implementing a new interpreter. Keep direct compiled evaluation as a screening or ranking facility unless its result is independently certified. In particular, `native_decide` has a different trust story from ordinary kernel reduction: its proof uses a compiler-related axiom, so it is not interchangeable with the current standard axiom profile [3, 4, 1].

After realizing a candidate, check its actual contract again and separately check that the intended executable definition compiles. A theorem about one kernel term is not automatically a correctness theorem for an arbitrary `implemented_by` replacement. An equality or simulation linking the published implementation to the certified term must be checked and recorded. If compilation fails, preserve the verified logical object but report the execution failure rather than silently changing the implementation.

7.2 R5.2. Does angelic execution compose with top-down search?

Answer. Yes as an over-approximate constraint semantics; the challenge is retaining all relevant alternatives efficiently. Top-down search contributes expected types, lexical contexts, and dependent constraints early. A bottom-up pool contributes already closed expressions, reusable behavior vectors, and a convenient finite representation for combinations. Moving from one organization to the other does not transfer a completeness or termination proof for free.

The appropriate top-down object is a typed hole together with a set of possible behaviors in the environments where it is observed. A recursive-call table must respect repeated calls to the same input and the result fiber for that input. An arbitrary independent answer at every occurrence is usually too weak; one fixed guessed answer with irreversible commitment is too strong. Maintain the constraint relating those answers, and backtrack the assumptions with the program branch.

The preservation condition and finite completeness statement of [proposition 5.2](#) apply here. An angelic success only schedules further work. A closed candidate is evaluated ordinarily. If it contradicts an assumed call result, reject that candidate or refine its assumption store without excluding other bodies that could satisfy the contract. If the call’s result was already a consequence of the contract, the stronger conditional refutation argument in [proposition 2.2](#) is available. These two sources of information must have different tags.

A small hybrid is preferable to replacing the full search. Enumerate shallow, closed terms bottom-up for a specific local telescope and expected type, then offer them as top-down alternatives. Reuse across contexts requires explicit renaming and a checked substitution. A behavior vector is an index into this pool, not permission to merge terms forever: later recursive calls can observe an input on which previously indistinguishable terms differ.

This recommendation uses the mechanisms discussed in R3 rather than assuming that the existing Burst implementation already proves the desired top-down algorithm correct. The gains or losses in memory, propagation power, and search coverage require the same typed benchmark and work budget on both variants.

7.3 R5.3. Is there a canonical small basis of recursion schemes with measured coverage?

Answer. There is no established canonical optimum for the combined Lean corpus specified here. A claim about coverage requires a fixed provider library, representation, cost bound, and meaning of “the same problem.” Existing recursive-synthesis corpora and systems are useful sources of tasks, but their success rates cannot be assigned to a new scheme basis without rerunning it. Trio is a relevant example of constructing recursive programs from nonrecursive expressions rather than merely enumerating raw recursor terms [24].

A defensible initial basis is the following. The examples describe expressibility, not newly measured synthesis success.

Scheme	Typical witness	Additional obligation
Single structural fold	List map, length; tree size	Constructor algebra and base value
Generalized parameter or function carrier	Accumulating reverse; left-fold behavior expressed by a right fold	Generalized induction hypothesis and finishing function
Tupled structural result	Fibonacci using a pair; count and sum together	Product algebra and projection
Multiple-argument recursion	Zip; merge with a decreasing total size	Generalization or a relation on the joint argument state
Course-of-values recursion	Calls to deeper, certified predecessors	Accessible predecessor capability and correct indexing
Well-founded measure	Euclidean-style recursion and decreasing search intervals	Well-foundedness and a decrease proof at each call

The rows overlap intentionally. Fibonacci does not intrinsically require course-of-values recursion: primitive recursion into a pair suffices. Zip can recurse on one list while inspecting the other. A flattening function may become easy once append is a permitted component, without inventing a new recursion scheme. These distinctions prevent attributing every failed body search to a missing recursor.

For each task record the cheapest sufficient scheme under a declared provider set, initially using a hand-written witness. Compare a carrier-given track, a scheme-given track, and a fully inferred track. The difference between their scores separates body-search difficulty from scheme invention. A proposed basis is successful if its measured coverage and cost improve; it need not be called canonical. Keep an explicit fallback to a larger bounded grammar so that a convenient initial basis does not become an undocumented completeness claim.

8 R6. Contracts, induction, and proof debt

8.1 R6.1. How should value holes and proof holes be interleaved?

Answer. Schedule a dependency graph of obligations, not two global queues called “program” and “proof.” A proof about unresolved code is sometimes useless, but sometimes it directly determines that code. Reflexivity can assign an index hole; an equality constraint can rule out a constructor before its arguments are filled; a type-class obligation can determine a type parameter. Therefore the blanket rule “proofs wait until all values are closed” loses important information.

Partition obligations by what they can do now. Eager obligations include well-scoping, type conversion, index constraints, required instances, and cheap local contradictions. Once their relevant value dependencies close, local branch contracts become eligible for a small proof service. Expensive global properties, induction choices, and speculative lemmas accrue proof debt and wait for a promising value skeleton. Every debt record carries its owning candidate, local telescope, dependencies, already tried proof tiers, and the snapshot or substitution under which it was created.

Synquid establishes the value of decomposing refinement specifications into component obligations [22]. The transferable idea is decomposition, not a requirement to put every Lean proposition into an external SMT solver. For LEANT2, start with a small collection of checked abstract facts: length expressions, numeric bounds, constructor information, and selected membership facts. Each transfer rule has a Lean theorem. A local abstract calculation then proposes a proof composition that the kernel checks. Unknown abstract facts stay unknown.

For example, synthesize vector-like list mapping together with length preservation using a motive of the form

$$M(xs) = \{ys : \text{List } B \mid |ys| = |xs|\}.$$

In the cons branch, the induction hypothesis supplies a recursive value ys and a proof $|ys| = |xs|$. Constructing $f(a) :: ys$ leaves only $1 + |ys| = 1 + |xs|$, which follows locally. There is no need to wait until a large closed recursor term is assembled before discovering the same induction.

This route is not a universal replacement for global verification. Involution, commutation of two calls, or preservation under concatenation may require a stronger invariant or an auxiliary lemma. The proof-debt graph should expose that difference rather than repeatedly spending the same local tactic budget.

A concrete integration issue is already visible: the current `proofPortfolio` rejects a target containing free variables, in addition to metavariables [1]. Thus the proposed local-branch proof service cannot be obtained merely by adding another tactic name to its existing list. It needs a context-aware wrapper, or a closed abstraction over the local telescope, with correct assignment and replay of the resulting proof. Test this explicitly on an induction hypothesis used inside a branch.

8.2 R6.2. Which properties admit induction, simplification, and arithmetic?

Answer. A useful sufficient class can be recognized, but there is no reason to expect an exact syntactic classifier for every property that some future induction-and-rewriting search might prove. Define the class by a certificate format rather than by an informal expectation of simplicity.

Fix a structural recursion scheme, an invariant template, a terminating rewrite policy, and a declared decidable theory for the remaining side conditions. A verification-condition generator creates one obligation per constructor. The certificate records the induction scheme, the invariant instance, the rewrite proofs, and the decision-procedure certificates. A successful translation into

this format establishes membership in the supported class. Failure to translate means unsupported, not false.

Proposition 8.1 (Local verification conditions suffice). *Let I be an inductively generated input type with a sound induction principle, and let f be defined by its structural recursion. Suppose that, for every constructor, the corresponding branch satisfies P assuming P for precisely the recursive arguments supplied by that induction principle. Then $\forall x : I, P(x, f(x))$.*

Proof. Apply the induction principle for I . At each constructor, unfold the defining equation for f and apply the certified branch obligation to the induction hypotheses. These are exactly the premises required by that obligation. $\square$

The substantive automation task is generating and proving the branch obligations, not the final induction theorem. Length preservation of map falls directly into this format. Reverse involution may need an append or accumulator invariant; sorting generally requires order and permutation lemmas. Such examples should be labeled by their additional certificate requirements rather than counted as unexpected failures of arithmetic.

Presburger arithmetic is a sensible index theory, but distinguish the theory from a particular tactic’s implementation and timeout. The current Lean manual states that `omega` does not yet implement every component of a full decision procedure, while describing a complete linear-integer-arithmetic procedure inside uncapped `grind`; capped invocations can still fail [4]. Therefore use the phrase “the registered, certified arithmetic fragment” in a completeness claim unless all required solver premises have actually been established. A timeout is an unresolved obligation.

The same idea supports a bounded invariant search: enumerate templates, generate finite branch conditions, and run exact fragment checkers when available. This is complete only for the template language and solver fragment, not for all true properties of structurally recursive programs.

8.3 R6.3. Which Lean induction facilities are usable now?

Answer. Assemble existing facilities around an explicit induction plan. Lean’s low-level induction routine is available in the pinned version, and the current reference also documents `fun_induction` for eligible structurally or well-founded recursive definitions, using function-specific induction theorems [2, 4]. These facilities reduce the cost of building a proof service, but do not themselves invent every required generalization or lemma.

The recommended adapter has three stages. First prepare the induction variable or recursive equation and a small ordered set of generalizations. Next obtain constructor or function-induction subgoals. Finally simplify computation rules and dispatch local obligations to an axiom-audited portfolio. Use direct assumptions, reflexivity, selected simplification, and decidable evaluation before larger arithmetic or saturation attempts. Library lookup belongs before repeated speculative induction when a known theorem already solves the goal.

Aesop supplies configurable proof search and normalization; `grind` combines its documented saturation and theory-solving facilities; premise selection and hammer tools can supply useful lemmas [8, 4, 26]. None of those facts supports a promise that arbitrary induction with helper-lemma invention is now solved in Lean. In particular, adding `grind` to the existing closed-goal portfolio does not replace choosing an induction hypothesis strong enough for the program.

For the implementation, keep version-specific tactic calls behind adapters. The exact presence and behavior of every optional tactic must be tested against the project lockfile, rather than

inferred from a moving online manual. The pinned induction and `cbv` entry points were source-verified here; no full integration build was performed. This separation permits a core-only default and optional external proof services without changing the acceptance gate.

An experiment on twenty or more quantified contracts should retain hand proofs classified as: direct simplification, arithmetic, structural induction, generalized induction, library lemma, or invented lemma. Report success and cost by class. An aggregate solved count conceals whether a new service actually addresses the intended induction bottleneck.

8.4 R6.4. Is there a principled policy for scheduling proof debt?

Answer. There is a principled optimization problem, but no justified claim that a standard independent-arm bandit theorem solves this particular scheduler. Proof attempts share lemmas, metavariables, counterexamples, and partially solved obligations. Their rewards change when a value choice changes. They also have switching costs from context setup and normalization. Treat expected benefit-per-cost as a heuristic, and keep fairness and acceptance semantics independent of its estimates.

For an eligible obligation o and proof tier t , a useful scheduling signal is

$$U(o, t) = \frac{\hat{p}_{o,t} V(o) + \lambda I(o)}{\hat{c}_{o,t} + s(o, t)}.$$

Here $\hat{p}$ estimates success probability, V estimates how much candidate work a proof unlocks, I estimates information useful for rejecting or refining a branch, $\hat{c}$ is work cost, and s is switching overhead. This formula is a proposed policy, not an optimality theorem. In particular, learning its parameters must not turn a failed prediction into a refutation.

A robust first scheduler uses deterministic rounds. Give every eligible obligation a cheap tier, select promising survivors for a larger tier, and age unselected obligations so that a resumable unbounded run does not starve them. Batch attempts that share a local context when the saved setup cost is material. Use logical work quanta, not a fixed millisecond ladder, for replay; expose wall clock as a separate interactive cutoff. Record attempted tiers so rollback does not repeat an identical expensive failure without a changed dependency.

Counterexamples are particularly valuable shared information. For a universally quantified contract, a closed, well-typed counterexample satisfying all preconditions can refute a candidate and become an early test for later ones. A malformed generated input, a failed premise, or an evaluator timeout cannot. The candidate still needs a proof of the whole universal contract before acceptance. This preserves the useful CEGIS discipline without confusing repeated testing with certification.

9 R7. Component retrieval at library scale

9.1 R7.1. What abstraction works for dependent, universe-polymorphic signatures?

Answer. Use a layered, constraint-carrying abstraction of *applications*, not an ordinary graph that connects each argument type separately to the result. An application requires all of its arguments. A provider $A \rightarrow B \rightarrow C$ does not make C constructible merely because an A is available. The appropriate first approximation is an AND-hypergraph or a resource model with conjunctive prerequisites. The accompanying finite model includes this elementary but consequential counterexample.

An abstract signature should retain the sort, head constant, binder arity, explicit/implicit/class roles, and sharing relationships between type variables. At a finer level it retains selected rigid indices and simple universe constraints. For example, the signature

$$(n : \mathbb{N}) \rightarrow \text{Vec } A \ n \rightarrow B$$

requires the vector’s index to agree with the chosen first argument. Keeping only the marginal heads `Nat`, `Vec`, and `B` deliberately forgets this relationship. Such forgetting may create a false positive, which concrete elaboration must reject; it must not silently justify an executable application.

Define sound abstraction relative to a fixed candidate language: every concrete application derivation allowed by that language must map to an abstract derivation. This implies that failure of exact abstract reachability excludes a concrete derivation, provided the abstraction and search bound correspond. Success remains only a proposal. Higher-order introductions, locally available functions, partial application, and dictionary construction must also be represented if they are allowed in the concrete language. An abstraction that models only first-order calls is not a sound exclusion oracle for a higher-order search.

Use refinement to restore information precisely where a candidate path failed: a shared index, a constructor of an indexed term, a universe relation, or a class prerequisite. Refuting one instantiation does not refute a provider for every instantiation. Keep refinement constraints scoped to the abstract combination or query that justified them. The general TYGAR architecture gives a well-motivated starting point, but the dependent refinement relation itself requires a new correctness argument [23].

Finally, distinguish *retrieval ranking* from *sound exclusion*. Taking the top twenty components is a heuristic reduction of the search language. It may be an excellent interactive choice, but it cannot justify a global negative claim about all imported declarations. A resumable exhaustive mode needs a fair route to the remaining permitted providers. An interactive receipt should name the actual provider set searched.

9.2 R7.2. Has type-guided abstraction refinement been combined with type classes?

Answer. Yes. This question has a direct answer in the proposal’s own cited Hoog+ paper. Its Sections 2.5 and 5.1 describe type-class support by dictionary passing: class constraints become extra dictionary arguments, class methods consume dictionaries, and instances supply or construct them. The described implementation does not cover arbitrary higher-kinded classes such as `Monad`; its supported fragment should not be confused with full Lean type classes [23].

For Lean, represent an obligation such as `[C A]` as an explicit prerequisite in the abstract application. A local instance is a currently available resource. A global instance declaration is a dictionary-producing component, possibly requiring other dictionaries. Concrete instance synthesis then checks that the prerequisite can actually be discharged in the current environment. A dictionary is not necessarily proof-irrelevant: classes can contain computational operations. Erasing all class arguments would lose behavior as well as applicability information.

The memoization key must include the class goal after relevant assignments, local instances and their priorities, the environment, and instance-search options. A cache indexed only by a printed pair “class, type” can be wrong in a changed local context. A type metavariable may also make an instance goal blocked rather than impossible; do not store that state as a permanent negative. Universe metavariables, output parameters, and recursive instance prerequisites are reasons to retain Lean’s actual constraint machinery behind the abstract index rather than

recreating resolution in the net.

Recommendation. Adapt the existing dictionary-passing idea, then measure where Lean’s extra constraints cause false positives. This is an adaptation problem with a clear prior solution for a smaller language, not a completely unexplored combination of abstraction refinement and type classes.

9.3 R7.3. Can premise selection use data goals and examples, and what corpus is suitable?

Answer. A data expression can certainly be supplied to a proposed ranking adapter, but the inspected proof-premise systems do not establish that such a query plus examples will produce good program-component rankings. LeanDojo and the Lean hammer provide relevant evidence for proof-oriented retrieval, not a published accuracy figure for this different task [25, 26].

Keep the interface small: a typed goal, local telescope, observation features, provider metadata, and a fixed maximum number of suggestions. For proposition goals, existing indexed lookup and proof-premise ranking are the natural first baseline. For data goals, use structural type fit, reachability, selected behavioral features, and only then learned ranking. A model trained to retrieve proof lemmas may be useful, but it needs a separately measured data-goal track.

Body-erased definitions from Std, Lean core, and selected Mathlib modules are a reasonable corpus source. Erasure must mean more than hiding the displayed body. If the target declaration remains available as a provider, the answer is already in the environment. Aliases, wrappers, an equation theorem that reconstructs the desired body, or a helper copied from the target can leak the solution too. The task harness should freeze an allowed provider set and audit its dependency closure against the erased target and forbidden reconstructions.

A hidden reference implementation may serve as an external observation oracle for the benchmark, but it must not appear in the synthesizer’s term-building context. Specifications should be published separately from that oracle. Record which library lemmas are intentionally permitted; otherwise a task may change silently from synthesis into lookup. Use separate tracks for provider-given, retrieval-required, and helper-invention problems.

Split train and test data by function families and dependency clusters, not only by declaration name. A test task that differs from a training task merely by namespace or one wrapper is poor evidence of generalization. Report provider recall at k , number of concrete applicability checks, dictionary-solving cost, and final certified-program success. Good retrieval recall can coexist with poor synthesis, and a fast synthesizer can mask weak retrieval on an overly small provider corpus.

10 R8. Ranking and equivalence

10.1 R8.1. Is there a defensible formal objective for ranking inhabitants?

Answer. Yes, as a declared preference model; no, as a uniquely correct ordering determined by an under-specified type and contract. Minimum description length is defensible because it makes a prior explicit. It does not remove the need to choose that prior, nor does it prove agreement with an individual user’s intent.

For a fixed, well-defined grammar distribution, one possible objective is

$$J(p) = -\log P(p \mid T, C, \mathcal{L}),$$

where $\mathcal{L}$ identifies the provider library and grammar version. The probability model must actually

assign probabilities to complete programs, or be an explicitly stated subprobability model, rather than merely attaching arbitrary weights and calling them probabilities. A code-length formulation can instead use an explicit prefix encoding of syntax and component names. Helper definitions must be charged; otherwise an arbitrarily complex solution can be hidden behind a newly invented short name.

The practical objective should respect a hierarchy. First enforce the requested type, contract, and axiom policy. Then distinguish executable from merely logical results where execution matters. Within accepted candidates, report program complexity, relevant library familiarity, estimated runtime, and proof cost separately before imposing a configurable or project-fixed ordering. A default order is a product decision; the report should not invent empirical user-preference evidence for it.

Unused-input counts are at most a soft feature. A projection or constant function can be exactly what the specification requests. Conversely, a term can mention an input in $x - x$ without making its output depend on that input. Syntactic occurrence is neither semantic information flow nor a correctness criterion. Penalizing all ignored induction hypotheses likewise disfavors legitimate predicates and short-circuiting traversals.

The present implementation ranks by unused binders, eliminator count, and raw expression size, then deduplicates by pretty-printed form [1]. Replace the last step independently of changing the preference model. Introduce an explicit total tie-break on a stable structural representation; the objective and tie-break belong in the receipt. Learning may change search priority while J remains frozen, as in R1. An optimality claim then refers to J and the covered grammar, not to an informal promise that the first program is what the user intended.

10.2 R8.2. Which equivalence is cheap and meaningful for dependent candidates?

Answer. Use a hierarchy of equivalence relations and label the level. Do not force one relation to serve kernel identity, pruning, display, and user explanation. The least expensive level is structural equality after safe normalization of binding names and metadata, with all relevant universe and type information retained. The next level is checked definitional equality or a certified normalization. A stronger level is an explicit theorem of equality, possibly pointwise equality of functions under the permitted axiom policy. Finite observation agreement is a separate, weaker classification.

For two functions, equality on a probe set says only that the probes do not separate them. It is useful for display: show one representative, the number of others, and a distinguishing input whenever one is found. It is not a proof of extensional equality. For dependent functions, probes and comparisons must be well-typed in each result fiber; comparing values across fibers requires a specified transport or heterogeneous relation.

Permanent observational pruning is justified only when the relation is a congruence for every continuation that the remaining search can build, with all future observations accounted for. This is a much stronger premise than agreement on the current examples. If a future recursive call or synthesized context can test a new input, preserve alternative representatives or refine the equivalence classes. The identity-versus-length-cutoff example in R2 illustrates exactly how finite agreement can fail to extend.

Proof irrelevance helps erase distinctions between proofs of the same proposition. It does not license dropping type dependencies, dictionaries that carry computations, or transport endpoints. For program-size ranking it is reasonable to count proof terms separately, but the checked term must keep the evidence necessary for typing.

Finally, pretty printing is a presentation function, not an injective encoding of terms. Hidden implicits, notation, and printing options can make distinct terms look the same. Use a canonical expression serialization with structural collision checks for exact deduplication. Use observation groups only for the user-facing summary unless the stronger congruence theorem has been established.

11 R9. Learned proposals and auditability

11.1 R9.1. What accuracy do models achieve on carrier and helper proposals?

Answer. The sources inspected for this report do not supply a reliable numerical answer for the exact task: a Lean type and examples, a fixed carrier/helper language, and an interactive end-to-end synthesis budget. Results for whole proof generation or premise selection are not substitutes for that measurement. The question is directly testable, and the benchmark should separate proposal quality from the downstream search’s ability to exploit a good proposal.

For each task, record an allowed carrier grammar, a known sufficient carrier or helper specification, the permitted components, and a reference implementation that is hidden from synthesis. Measure at least four successive events: the proposal parses; it is well-typed and within the declared grammar; it enables a solution under a fixed body-search work budget; and that solution passes the full contract gate and execution checks. The first two are cheap interface successes, not evidence of semantic success.

Compare four arms with the same total budget: the ordinary ordered grammar; a fixed learned reordering of that grammar; model proposals restricted to the grammar; and proposals allowed to expand it. The fourth arm changes the search language and must be reported separately. Include a carrier-given oracle arm to estimate how much headroom is actually attributable to carrier choice rather than body search, proof automation, or unavailable components.

Report success at a fixed proposal count and at a fixed total cost. A top-ten proposal list is not free if each item triggers an expensive search. Include model inference time, rejected proposal processing, compilation and proof costs, and time to first certified solution. Record exact model identifiers, prompts, provider sets, seeds where available, and raw outputs. Use held-out function families, several example sets per task, and uncertainty intervals clustered by task family. Avoid treating many nearly identical variants as independent observations.

The resulting study could justify a model lane without making it part of the trusted logic. Until it is performed, the appropriate answer is an evaluation protocol, not an invented percentage. This report’s Python checks contain no model-accuracy experiment and make no claim about one.

11.2 R9.2. What determinism and auditability are possible beyond recording proposals?

Answer. Separate certificate replay, search replay, and model regeneration. A closed accepted program and proof can be rechecked without reproducing any search or model output. Replaying the same search additionally requires the logical schedule and all proposal inputs. Regenerating identical model outputs requires a controlled inference implementation and environment; a fixed seed or temperature zero alone is not a sufficient specification of that system.

Use a restricted proposal protocol: a typed carrier expression, a list of local variables to generalize, a helper type, or references to permitted component names. Elaborate these in a frozen environment under explicit size and resource limits. Do not accept an arbitrary tactic, import command, process invocation, or metaprogram merely because it arrived in the same text field as

a type. The model chooses data for the synthesis engine; it does not choose the engine’s trust policy or acquire permission to execute arbitrary code.

A reproducible proposal record should contain the query and environment fingerprints, grammar and provider versions, prompt bytes, model identifier, inference settings, raw response bytes, parsed proposal representation, validation outcome, and the logical point at which the proposal was admitted. A stable canonical representation is useful for comparison, but preserve the original bytes for auditing parser and elaborator behavior. Record absence or cancellation of a proposal as an input event too.

If suggestions only reorder alternatives within a finite grammar and the scheduler remains fair, the bounded completeness argument can be retained. They can still change which candidate is discovered before an interactive deadline. A model that proposes an out-of-grammar helper changes the grammar; no receipt should describe that run as exhausting the original grammar under an unchanged meaning of failure.

For a core-only default, the same proposal interface can be implemented by a small deterministic grammar. An optional model is then a replaceable proposer, not an architectural dependency. The acceptance receipt cites the checked program, contract proof, and axiom inventory. Model output is useful provenance, not a premise in the mathematical proof of correctness.

12 An integrated architecture and implementation plan

12.1 Six interfaces with explicit trust boundaries

The recommendations above can be implemented without a single monolithic “intelligent search” subsystem. Six interfaces are enough to make the critical contracts visible.

Frozen query. Contains the target and contract, local telescope or its closed abstraction, imports and environment identity, axiom policy, provider set, grammar version, ranking objective, and operational bounds. A changed provider set or equality policy creates a new query identity even when the printed target is unchanged.

Persistent search node. Contains the relevant Lean saved state, ordered obligations, dependency summary, consumed recursion resources, residual assumptions, ranking lower bound, and stable decision-path identity. Global work accounting stays monotone outside rollback; branch-local semantic data either rolls back or is guarded by complete read-dependency keys.

Proposal service. Returns typed alternatives with provenance and a declared grammar effect. Grammar enumeration, library retrieval, failed-proof generalization, and model suggestions are implementations of this interface. The service may be wrong; the consumer validates its output.

Observation service. Returns a typed observation result, dependencies, and either a conservative refutation justification, a checked proof, or an unknown/residual constraint. Contract consequences and speculative angelic assumptions have different constructors. An evaluation error cannot masquerade as a Boolean false.

Proof service. Receives an obligation in a specific telescope and snapshot, a permitted axiom profile, and a work quantum. It returns a checked proof, a validated counterexample when

appropriate, an unsupported-fragment report, or an inconclusive resource result. It does not return a bare Boolean whose meaning mixes these cases.

Acceptance and publication. The existing gate is the starting point for type, proof, and axiom checks. Publication additionally verifies the executable representation when one is promised. A receipt carries the exact certified term and enough provenance to recheck it without trusting search, caches, models, or pretty printers.

12.2 A proposed control loop

The following is architectural pseudocode, not a compiled Lean implementation. Its important feature is the placement of semantic decisions.

```
freeze query, grammar, providers, trust policy, ranking objective
frontier := [initial well-scoped search node]
while frontier is not empty and logical budget remains:
  node := select by frozen epoch policy and stable tie-break
  restore node's saved Lean state
  validate or invalidate node's cached dependency records
  discharge eligible cheap constraints
  if there is a certified conservative refutation:
    discard this branch with its justification
  else if program and required proof are complete:
    run acceptance gate
    if accepted: record certificate; test publication separately
  else:
    choose an eligible obligation from the dependency graph
    request bounded proposals
    validate each proposal in a transaction
    enqueue resulting states, including the fair fallback
  commit learning and parallel results only at epoch boundaries
return accepted certificates plus exact coverage/stop metadata
```

Actual wall-clock cancellation can interrupt this loop or an individual service. It must be reported separately from logical exhaustion. Long kernel or tactic operations require their own interruption or heartbeat limits; checking the clock only when entering a search node is not a hard bound on total runtime. The current engine disables its outer heartbeat limit and uses explicit lane deadlines, so this interaction deserves an integration test [1].

12.3 A revised sequence of milestones

The proposal's research order is broadly sensible, but some low-risk reuse should move earlier and some acceptance gates should change. In particular, receipts, negative-result semantics, and rollback-safe cache identities are prerequisites for trustworthy experimentation, not merely finishing features.

Phase	Implementation scope	Exit evidence
P0	Freeze source and benchmark manifests; define receipt, stop reasons, cache ownership, and trust profiles	Recheck saved certificates without search; distinguish unsupported, interrupted, grammar-exhausted, and proved-impossible

Phase	Implementation scope	Exit evidence
P1	Separate logical acceptance from executable publication; indexed provider lookup; guarded Nat/indexed induction; compare existing certified evaluation	Compile/evaluate raw-recursion replacements; targeted probes; same bounded semantic outcomes; measurements including failed attempts
P2	First-class whole-state frontier, stable ordering, logical budgets, dependency summaries, and conservative memoization	Exhaustive small-grammar comparison against the reference search; replay identical decision traces; rollback adversarial tests
P3	Typed per-observation reports, certified decompositions, guarded incremental caches, local proof obligations	Cache-on/cache-off semantic equivalence; all fragment refutations justified; local induction-hypothesis tests
P4	Bounded carrier packages, conflict-graph lower bounds, recursion capabilities, helper/generalization proposals	Carrier-given versus invented tracks; finite completeness checks; interpretation/realization proof obligations
P5	Richer motives and transport policies, inductive contracts, refined library reachability	Indexed and quantified-contract coverage; provider recall; explicit limits of each supported fragment
Optional	Parallel speculative work and learned proposal sources behind the same interfaces	Deterministic epoch replay; end-to-end latency/cost ablations; no change in certificate acceptance semantics

Nat recursion and basic guards need not wait for an invented minimal-carrier theory. Equally, a custom evaluator should not be justified merely by the fact that residual evaluation was expensive on one historical query. Measure the existing proof-producing evaluator and finer observation decomposition first. Retrieval of a missing standard lemma can be independent of the carrier work.

The acceptance criterion “identical candidate lists under equal wall time” should be removed from the search-rewrite phase. It is neither a meaningful semantic equivalence test nor stable across machines. Instead compare complete candidate sets on a finite grammar, first results under a fixed logical trace, and independently measured latency distributions. Ranking changes should have a separate benchmark and explicit version.

12.4 Engineering items that still need semantic care

The engineering/research distinction is useful, but the proposal’s engineering items are not exempt from correctness obligations. E1 must link a compilable presentation to the certified program. E2 must preserve local scoping and expected-type constraints when turning a sketch into goals. E3 must record the actual environment and choices, not just a printed answer. E4 needs a profile that attributes work without counting nested timers twice. E5 needs deterministic commit rules, rather than a race followed by sorting. E7 must preserve trust profiles when adding tactics, and E8 must prevent benchmark leakage.

E6 has an additional abstraction issue. A propositional decision procedure over independent atoms can produce an intuitionistic proof or a countermodel for the *reified formula*. That countermodel need not refute the original Lean goal when the chosen atoms are concrete propositions

related by arithmetic or contextual hypotheses. For example, treating a theorem of arithmetic as an unconstrained atom can make an unprovable propositional abstraction of a provable Lean statement. Thus report “not derivable in this propositional abstraction,” unless an interpretation theorem transfers the countermodel to the actual query. A kernel-checked proof of $T \rightarrow \text{False}$ remains a fundamentally different artifact.

Finally, “no settings” for an ordinary user does not mean “no configuration identity.” Defaults, imported attributes, grammar bounds, axiom profiles, and model versions all affect behavior. Keep the interactive surface simple while making those choices explicit in receipts and benchmark manifests.

13 Validation, experiments, and limits of the evidence

13.1 What was executed for this report

The accompanying `experiments/model_checks.py` is a deterministic, standard-library-only Python program. All eight groups of assertions passed. It explores deliberately small models of the semantic distinctions above; it is neither a port of LEANT2 nor a Lean kernel verification.

Check	Executed result	What it establishes
Coupled-goal cost	Two obligations close in one action of cost 1; summing unit bounds gives 2	Counterexample to the unqualified open-goal sum heuristic
Reverse fold	All 255 binary lists of length at most 7 agree with reverse	Finite validation of the list-carrier implementation, not a complexity claim
Incomplete rows	Three distinct partial rows have only one conflict edge; chromatic number 2	Distinct partial rows cannot be counted as distinct states
Finite carriers	Exhaustive parity and unary length-modulo-three tables; minimum sample-consistent state counts 2 and 3	Exact minima in the declared finite table models
Rollback cache	Hole-only cache returns stale value 0; branch-versioned cache returns expected value 1	A minimal example of the need for assignment identity
Parametric probes	Two uniform list functions agree at lengths 0 through 4 and differ at length 5	Finite length coverage does not prove universal equality
Deadline versus work	Simulated work limits 10 and 20 discover different sets; fixed work 10 gives the same set	Sorting cannot repair different discovery sets
Conjunctive retrieval	An ordinary path reaches C from A for a provider requiring A and B ; the hypergraph does not	Application prerequisites must be conjunctive

The finite-carrier enumeration is small enough to inspect exactly. It fixes the alphabet and output sets and enumerates all labeled transition tables, all seeds, and all finalizers at each stated state count.

Sample task	Samples	States	Tables checked	Solutions
Binary parity, length ≤ 4	31	1	2	0
	31	2	128	2
Unary length modulo 3, length ≤ 5	6	1	3	0
	6	2	72	0
	6	3	2187	6

The 2392 table checks confirm the executable finite procedure in R2. The first parity witness uses seed 0, finalizer (0, 1), and transitions that leave the state unchanged on symbol 0 and exchange it on symbol 1. The first unary witness cycles $0 \mapsto 1 \mapsto 2 \mapsto 0$ and returns the current state. The script’s solution counts include different labelings; it does not quotient machines by isomorphism.

The rollback check is a model counterexample, not a reproduced defect in the actual Lean implementation. Likewise, the deadline simulation uses logical work positions, not measured processors or seconds. These distinctions are part of the results file itself, not qualifications added only to this article. Run the checks from the archive root with:

```
python3 experiments/model_checks.py --output experiments/results.json
```

13.2 The required Lean regression and ablation suite

A production implementation should add tests at three different levels. First, semantic unit tests validate individual rules: scope preservation, metavariable isolation, context substitution, contract decomposition, conservative false results, transport reconstruction, and recursive-call capabilities. Every cache should have a cache-disabled reference mode.

Second, exhaustive bounded tests compare the original and refactored search on small finite provider and syntax sets. Compare typed candidate sets before ranking. Exercise shared universe variables, delayed assignments, changed local instances, identical-looking goals in different telescopes, and rollback to a branch that reuses an identifier with a different assignment. The acceptance gate does not detect a valid candidate that an unsound prune silently discarded, so positive-result testing alone is inadequate.

Third, end-to-end benchmarks measure usefulness. Preserve the existing baseline as historical regression material, but add indexed families, Nat recursion, conditional programs, carrier-given and carrier-invented Church problems, quantified contracts, and retrieval-required definitions. Each entry records its source, permitted providers, examples, universal contract where available, reference proof or implementation, and leakage controls.

Use both a logical-work track and a wall-clock track. The first tests replay and coverage; the second tests responsiveness at declared budgets such as 1, 5, 10, and 60 seconds. Those budgets are proposed measurement points, not results of this report. Distinguish cold environment/index loading from a warmed session. Report time to the first certified candidate, number and rank of distinct behavioral groups, proof and compilation cost, peak memory, and unresolved versus refuted outcomes.

An ablation should change only one mechanism at a time: fine-grained observations, existing cbv, guarded caching, dependency-aware goal order, carrier proposals, retrieval, or learned priorities. The proposal’s historical 45% residual-evaluation figure is motivation to measure evaluation, not proof that replacing that evaluator improves all queries [1].

13.3 Claims that this investigation does not make

No Lean executable was available in the execution environment, so no revised Lean engine, tactic wrapper, or example program was compiled here. The pinned repository and selected Lean source files were inspected through their source interfaces. The repository’s reported benchmark scores remain its historical measurements, not independently reproduced results.

The propositions and theorem in this report are mathematical arguments with explicit hypotheses, not machine-checked Lean theorems. Their small-model instances were tested where stated. In particular, the dependency analysis and interpreter simulation still require implementation-specific proofs or checked certificates. No new asymptotic complexity bound for unrestricted dependent synthesis, no universal minimal-carrier algorithm, and no model-accuracy number is claimed.

Bibliographic searches establish positive evidence for the cited mechanisms. They do not establish that no other paper has addressed an adjacent problem. Statements that a source does not answer a question refer to the inspected sources and the precisely stated task. This is especially important for higher-order dependent live evaluation, automated generalization, and latency-bounded model proposals, where neighboring research can use a different language or contract model.

14 Conclusion

The central expert answer is that LEANT2 does not need one breakthrough that solves all nine research areas simultaneously. It needs explicit boundaries between an accepted program, a conservative search restriction, a bounded negative result, and a preference among solutions. Once those boundaries are fixed, several apparently open implementation questions have immediate starting points in Lean and the existing synthesis literature.

Whole-state snapshots provide a sound baseline for a dependency-aware search; reproducible logical epochs provide a meaningful alternative to impossible machine-independent deadline promises. Finite carrier packages can be searched exactly, and witnessed incompatibility gives a sound lower bound without pretending that incomplete observation rows form a known minimal automaton. Typed hole closures, guarded incremental caches, and equation-based certified evaluation support stronger pruning without treating a final kernel gate as a repair for lost solutions. Motive computation, indexed induction, and dictionary constraints have existing machinery to reuse. Program-and-proof synthesis can start from local verification conditions instead of waiting for universal lemma invention.

The recommended next artifact is therefore an auditable bounded engine with strong small-instance tests, not an unqualified “general synthesis” claim. Its research extensions can then be evaluated independently: richer carrier proposals, better generalization, dependent residual constraints, and optional learned guidance. Each improvement has a concrete success criterion, a known failure mode, and a receipt that states exactly what was checked.

A Source map and question coverage

The numbering R1.1–R9.2 in this report follows the order of the questions in the proposal, not a new selection of topics. The following map makes it possible to compare the answers directly with the original source at the pinned commit. Paths in the first nine rows are relative to `docs/proposals/11-further-improvements/sections/`.

Source	Questions covered	Report section
05-search.tex	R1.1–R1.4: independence, lower bounds, online costs, wall-clock determinism	3, p. 6
06-carriers.tex	R2.1–R2.5: minimal carriers, polymorphism, fusion, recursor motives, generalization	4, p. 9
07-evaluation.tex	R3.1–R3.4: higher-order/dependent holes, incrementality, verified evaluation, angelic completeness	5, p. 14
08-dependent.tex	R4.1–R4.4: motive enumeration, transport, transparency, Lean API	6, p. 17
09-recursion.tex	R5.1–R5.3: realization, top-down angelic execution, scheme basis	7, p. 19
10-contracts.tex	R6.1–R6.4: proof/value scheduling, inductive fragment, Lean automation, proof debt	8, p. 22
11-retrieval.tex	R7.1–R7.3: dependent abstraction, type classes, program-component relevance	9, p. 24
12-ranking.tex	R8.1–R8.2: ranking objectives, equivalence	10, p. 26
12-ranking.tex	R9.1–R9.2: model proposal accuracy, auditability	11, p. 28
Leant2/Search/Core.lean	Search rules, residual evaluation, induction filter, proof portfolio	Sections 3–9
Leant2/Native/Transaction.lean	Saved-state boundary, global ledger, blocker cache, deadlines	Sections 3, 5
Leant2/Engine.lean	Candidate ranking, grace period, contract extraction	Sections 3, 10
Leant2/Accept/Gate.lean	Kernel checking and transitive axiom audit	Sections 2, 12
Leant2/Core/Types.lean	Trust profiles and result algebra	Sections 2, 12

The proposal’s evidence, engineering, adaptation, roadmap, expert-summary, and bibliography sections were also reviewed. In particular, this report does not treat the proposal’s explicitly memory-based bibliography as already verified: the relevant positive claims were checked against the primary sources listed below.

References

- [1] Vladimir Reshetnikov. *Leant2 repository*, snapshot 784664ff34e819422510a4d470ab07fe48bb736b, 18 September 2026. *Leant2: Design for Further Improvements* and implementation files mapped in Appendix A. Accessed 21 September 2026. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b>.
- [2] Lean contributors. *Induction metaprogramming implementation*, Lean 4.34.0. Source-level API inspection, accessed 21 September 2026. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [3] Lean contributors. *Call-by-value tactic frontend*, Lean 4.34.0. Includes `cbv` and `decide_cbv`. Accessed 21 September 2026. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Elab/Tactic/Cbv.lean>.
- [4] Lean contributors. *The Lean Language Reference: Tactic Reference*. Moving online reference, accessed 21 September 2026. Capability descriptions from the current reference are distinguished from pinned source checks. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [5] Lean contributors. *The Lean Language Reference: Recursive Definitions*. Accessed 21 September 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [6] Lean contributors. *Theorem Proving in Lean 4: Inductive Types*. Natural-number recursion and addition. Accessed 21 September 2026. https://lean-lang.org/theorem_proving_in_lean4/Inductive-Types/.
- [7] Jannis Limperg and Asta Halkjær From. Aesop: White-Box Best-First Proof Search for Lean. *CPP*, 2023. <https://doi.org/10.1145/3573105.3575671>. Author-deposited record: <https://zenodo.org/records/7430233>.
- [8] Aesop contributors. *Aesop README*, accessed 21 September 2026. Source blob f75ec1f8bebcf6d857fc3954cf91cf99f323a758. <https://github.com/leanprover-community/aesop/blob/master/README.md>.
- [9] Chase Norman and Jeremy Avigad. Canonical for Automated Theorem Proving in Lean. *ITP*, 2025. Inspected preprint version 1. <https://arxiv.org/html/2504.06239v1>.
- [10] Shraddha Barke, Hila Peleg, and Nadia Polikarpova. Just-in-Time Learning for Bottom-Up Enumerative Synthesis. *OOPSLA*, 2020. <https://arxiv.org/abs/2010.08663>.
- [11] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47(4), 173–180, 1993. <https://www.sciencedirect.com/science/article/pii/0020019093900299>.
- [12] Jeremy Gibbons, Graham Hutton, and Thorsten Altenkirch. When is a Function a Fold or an Unfold? *Electronic Notes in Theoretical Computer Science* 44(1), 146–160, 2001. <https://www.cs.ox.ac.uk/publications/publication2368-abstract.html>.
- [13] Tjark Weber and James Caldwell. Constructively Characterizing Fold and Unfold. *LOPSTR 2003*, LNCS 3018, 110–127. https://doi.org/10.1007/978-3-540-25938-1_11.
- [14] Jean-Philippe Bernardy, Patrik Jansson, and Koen Claessen. Testing Polymorphic Properties. *ESOP*, LNCS 6012, 125–144, 2010. https://doi.org/10.1007/978-3-642-11957-6_8.
- [15] Dana Angluin. Learning Regular Sets from Queries and Counterexamples. *Information and Computation* 75(2), 87–106, 1987. [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6).
- [16] Justin Lubin, Nick Collins, Cyrus Omar, and Ravi Chugh. Program Sketching with Live Bidirectional Evaluation. *Proceedings of the ACM on Programming Languages* 4(ICFP), article 109, 2020. <https://doi.org/10.1145/3408991>. Preprint: <https://arxiv.org/abs/1911.00583>.

- [17] Niek Mulleners, Johan Jeuring, and Bastiaan Heeren. Program Synthesis Using Example Propagation. Preprint, 2022. <https://arxiv.org/abs/2210.13873>.
- [18] Umut A. Acar, Matthias Blume, and Jacob Donham. A Consistent Semantics of Self-Adjusting Computation. Preprint, 2011. <https://arxiv.org/abs/1106.0478>.
- [19] Mario Carneiro. Lean4Lean: Verifying a Typechecker for Lean, in Lean. Preprint, version 3, revised 14 September 2025. <https://arxiv.org/abs/2403.14064v3>.
- [20] Anders Miltner, Adrian Trejo Nuñez, Ana Brendel, Swarat Chaudhuri, and Isil Dillig. Bottom-Up Synthesis of Recursive Functional Programs using Angelic Execution. *Proceedings of the ACM on Programming Languages* 6(POPL), article 21, 2022. <https://doi.org/10.1145/3498682>. Preprint: <https://arxiv.org/abs/2107.06253>.
- [21] Łukasz Czajka. Practical Proof Search for Coq by Type Inhabitation. *IJCAR*, 28–57, 2020. https://doi.org/10.1007/978-3-030-51054-1_3. Author page: <https://www.mimuw.edu.pl/~lukaszcz/sauto/index.html>. Unfolding options checked in the project documentation: <https://coqhammer.github.io/>.
- [22] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *PLDI*, 2016. <https://arxiv.org/abs/1510.08419v3>.
- [23] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *Proceedings of the ACM on Programming Languages* 4(POPL), article 12, 2020. <https://doi.org/10.1145/3371080>. Inspected author PDF: https://ranjitjhala.github.io/static/hoogle_plus.pdf.
- [24] Woosuk Lee and Hangyeol Cho. Inductive Synthesis of Structurally Recursive Functional Programs from Non-recursive Expressions. *Proceedings of the ACM on Programming Languages* 7(POPL), article 70, 2023. <https://doi.org/10.1145/3571263>.
- [25] Kaiyu Yang et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. *NeurIPS*, 2023. <https://arxiv.org/abs/2306.15626>.
- [26] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Q. Jiang, and Sean Welleck. Premise Selection for a Lean Hammer. Preprint, 2025. <https://arxiv.org/abs/2506.07477>. Author project page: <https://cmu-13.github.io/lean-hammer/>.